

Dataset and Preprocessing Description

GAN-Based Synthetic Handwritten Digit Generation · 102903/C0701B Deep Learning ·

Rajagiri School of Engineering & Technology

Overview

This document describes the dataset used for the project and the preprocessing pipeline applied to it before training. The model is a Generative Adversarial Network trained to generate synthetic handwritten digit images, and the dataset used for this task is MNIST.

Dataset Description

Origin and composition

MNIST, the Modified National Institute of Standards and Technology database, is a collection of handwritten digit images assembled by Yann LeCun, Corinna Cortes, and Christopher J. C. Burges in 1998. It was built by remixing two earlier NIST collections: Special Database 1, which contains digits written by American high school students, and Special Database 3, which contains digits written by employees of the United States Census Bureau. The original NIST sets were combined and reprocessed so that the training and test splits each draw from a mix of writers, which keeps the handwriting style in both splits representative rather than skewed toward one population.

Each digit in the original NIST collections was isolated, size-normalized to fit within a 20 by 20 pixel box while preserving its aspect ratio, and then centered in a 28 by 28 pixel field using the center of mass of the pixel values. This centering and normalization was done once by the dataset's creators, so every image supplied by MNIST already arrives in this standard form. MNIST has served as the standard benchmark for handwritten digit recognition and generative modeling since its release, precisely because this preparation removes positional and scale variation that would otherwise have to be handled separately.

Structure and Format

Property	Value
Name	MNIST (handwritten digits)
Reference	LeCun, Y., Cortes, C., and Burges, C. J. C. (1998). The MNIST Database of Handwritten Digits

Property	Value
Training set size	60,000 images
Test set size	10,000 images
Image dimensions	28 x 28 pixels
Color channels	1 (greyscale)
Pixel depth	8-bit (values 0 to 255 in the raw files)
Classes	10, corresponding to the digits 0 through 9
Native file format	IDX (ubyte), the format used in the original release
Total size on disk	Approximately 11 MB compressed

Class distribution and image characteristics

The ten digit classes are approximately, though not perfectly, balanced: each digit accounts for roughly 9 to 11 percent of the training set. Every image is a single-channel, anti-aliased grey-scale bitmap, so pixel intensities vary smoothly across a stroke edge rather than being purely black or white. No image carries color, background texture, or additional metadata beyond its class label. Because the digits were already centered and size-normalized by the dataset's creators, no manual cropping or alignment was required for this project.

Access method for this project

The dataset is not stored in the project repository. It is downloaded automatically the first time the training script runs, through the following call:

```
train_dataset = torchvision.datasets.MNIST(
    root="./data", train=True, download=True, transform=transform
)
```

Only the training split, all 60,000 images, is used. The test split is not used, since the project is a generative modeling task rather than a classification task with a held-out evaluation set. The `download=True` flag fetches the files into a local `data/` folder on first run and reuses that local copy

on every run after, so the dataset never needs to be added to version control. The `data/` folder is listed in the project's `.gitignore` for this reason.

Preprocessing Pipeline

Four preprocessing steps are applied to the raw MNIST images before they reach the Generator and Discriminator networks. Each step corresponds directly to a line in `train_gan_simple.py`.

Step 1: Loading images as tensors

Each image is read from the MNIST files and converted from its native format into a PyTorch tensor using `torchvision.transforms.ToTensor()`. This conversion reshapes each image into a tensor of shape (1, 28, 28), one channel by height by width, and rescales the raw 8-bit pixel values from the integer range 0 to 255 into a floating-point range of 0 to 1. This is a standard first step for feeding image data into a neural network, since networks expect floating-point input rather than raw integer pixel values.

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5])
])
```

Step 2: Normalizing pixel values

The tensor's pixel values are then rescaled a second time, from the 0 to 1 range produced by `ToTensor()` into a range of negative 1 to positive 1, using `transforms.Normalize([0.5], [0.5])`. This operation subtracts 0.5 and divides by 0.5 for every pixel. The reason for this second rescaling is architectural: the Generator network ends with a `Tanh` activation function, which produces outputs in the range negative 1 to positive 1. If the real images the Discriminator sees were left in the 0 to 1 range while generated images arrived in the negative 1 to positive 1 range, the Discriminator could learn to tell real and fake images apart just by their value range rather than their content. Matching the two ranges removes that shortcut.

Step 3: Batching and shuffling

The normalized dataset is wrapped in a PyTorch `DataLoader` with a batch size of 128 and shuffling enabled.

```
loader = torch.utils.data.DataLoader(
    dataset, batch_size=128, shuffle=True
)
```

A batch size of 128 balances training speed against how many images contribute to each gradient update. Shuffling is enabled so that the order in which images are seen changes every epoch. This prevents the Discriminator from learning any pattern tied to the fixed order of the dataset and gives the Generator a more varied and representative sample of digits within every batch.

Step 4: No augmentation applied

No augmentation, such as random rotation, flipping, or cropping, is applied to the images. This was a deliberate choice rather than an oversight, for two reasons. First, MNIST already contains 60,000 training images with substantial natural variation in handwriting style, which is enough data for a project at this scale without needing to synthesize more. Second, common augmentations such as flipping would distort digit identity: a flipped 2 or 7 does not correspond to natural handwriting variation, and could teach the model incorrect digit shapes. Rotation and cropping were avoided for a related reason, since MNIST digits are already centered and size-normalized, and disturbing that alignment would work against the benefit the dataset already provides.

Summary: Raw Data Versus Preprocessed Data

Property	Raw MNIST image	After preprocessing
Data type	8-bit unsigned integer	32-bit floating point tensor
Shape	28 x 28, single channel	(1, 28, 28)
Pixel value range	0 to 255	negative 1 to positive 1
Batch structure	Individual images	Batches of 128, reshuffled every epoch
Order	Fixed dataset order	Randomized each epoch

Reproducibility

The random seed is fixed at 42 across Python's `random` module, NumPy, and PyTorch (`torch.manual_seed`) before the dataset is loaded. This keeps the shuffling pattern and any other randomness in the data pipeline consistent across repeated runs, so the preprocessing steps described above behave identically each time the training script is executed.